

Metodología de Sistemas II

Trabajo Práctico 1: SOLID

Farias, Gustavo
Matrícula: 101662
Comisión: M2025-13

Repositorio GitHub: [Lucenear/Tecnicatura_Programacion_UTN](https://github.com/Lucenear/Tecnicatura_Programacion_UTN)

Parte A - Diagnóstico del Diseño Actual

1. Principios SOLID Violados

- SRP (Responsabilidad única): La clase Notificador tiene múltiples responsabilidades. No solo orquesta el envío, sino que también conoce la lógica de formato y transmisión de cada canal (Email, SMS, WhatsApp). Si cambia el protocolo de SMS, se debe modificar esta clase.
- OCP (Abierto / Cerrado): El diseño está cerrado a la extensión pero abierto a la modificación. Para agregar un nuevo canal (por ejemplo Telegram), es obligatorio abrir la clase Notificador y agregar un nuevo bloque if. Esto viola directamente OCP.
- DIP (Inversión de dependencias): Notificador depende de implementaciones concretas ("EMAIL", "SMS") en lugar de abstracciones. No hay ninguna interfaz o contrato que desacople al gestor de turnos del mecanismo de notificación específico.

2. Fragilidad del Diseño

El diseño es frágil porque cualquier cambio en un canal de notificación afecta a todos los demás al compartir la misma clase. Un error tipográfico en un nuevo if puede romper la lógica de canales existentes. Además, el uso de cadenas ("EMAIL", "SMS") como discriminadores de tipo no ofrece seguridad en tiempo de compilación; un valor incorrecto simplemente no envía nada, sin generar error visible.

3. Problemas de Mantenibilidad

- Acoplamiento alto: La lógica de negocio (gestión de turnos) estará inevitablemente acoplada a los detalles de infraestructura de cada canal.
- Complejidad creciente: Cada nuevo canal agrega otro bloque condicional, haciendo que el método enviar crezca linealmente y sea más difícil de leer y testear.
- Imposibilidad de testing unitario aislado: No se puede probar el envío de email sin instanciar toda la clase Notificador, incluyendo la lógica de SMS y WhatsApp.

4. Impacto en el Costo del Cambio

Agregar un nuevo medio de notificación requiere:

- Modificar una clase existente (Notificador).
- Agregar lógica condicional nueva.

- Re-testear todos los canales existentes por riesgo de regresión.
- Desplegar todo el módulo de notificaciones aunque solo cambió un canal.

Esto convierte un cambio de bajo impacto en uno de alto riesgo y costo, escalando linealmente con la cantidad de canales soportados.

Parte B y C — Refactor Aplicando SOLID + Extensibilidad

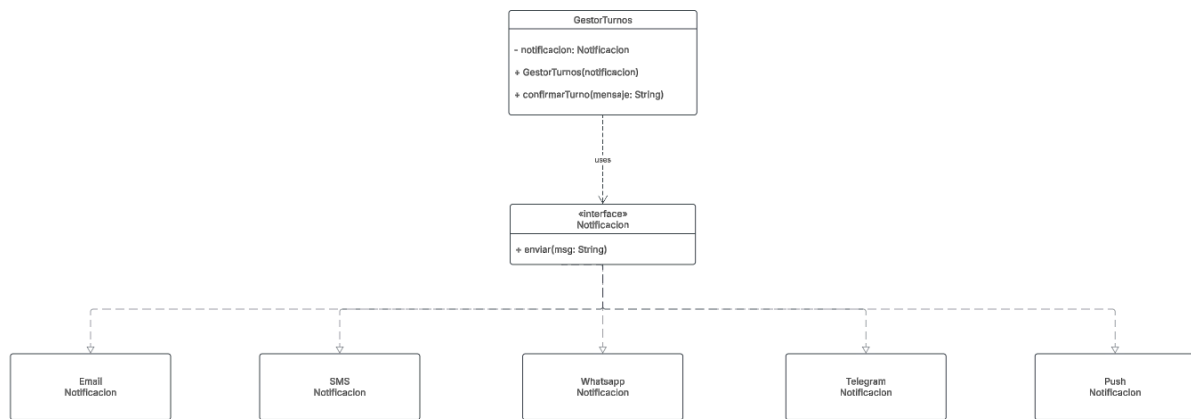
El nuevo diseño aplica SOLID de la siguiente manera:

- SRP: Cada clase (EmailNotificacion, SmsNotificacion, etc.) tiene una única responsabilidad: saber cómo enviar mensajes por su canal específico.
- OCP: Agregar TelegramNotificacion o PushNotificacion requiere solo crear una nueva clase que implemente Notificacion. Cero modificaciones a clases existentes.
- LSP: Cualquier implementación de Notificacion puede sustituirse por otra sin alterar el comportamiento esperado del sistema. El contrato enviar(String mensaje) es uniforme.
- ISP: La interfaz Notificacion es mínima y específica. Ninguna clase implementadora está forzada a depender de métodos que no usa.
- DIP: GestorTurnos depende exclusivamente de la abstracción Notificacion, nunca de implementaciones concretas.

Parte D — Dependency Inversion Principle

La clase GestorTurnos recibe la dependencia Notificacion mediante inyección por constructor. Esto permite:

- Cambiar el canal de notificación sin tocar GestorTurnos.
- Inyectar mocks/stubs en pruebas unitarias.
- Configurar el canal en tiempo de ejecución (por ejemplo, según preferencia del paciente).



Creacion de la clase GestorTurnos.java

```

public class GestorTurnos {

    // DIP: Dependemos de la abstraccion Notificacion
    private final Notificacion servicioNotificacion;

    // Constructor
    public GestorTurnos(Notificacion servicioNotificacion) {
        this.servicioNotificacion = servicioNotificacion;
    }

    public void confirmarTurno(String mensaje) {
        servicioNotificacion.enviar(mensaje);
    }

}
  
```

Código Java:

Notificacion.java

```

public interface Notificacion {
    void enviar(String mensaje);
}
  
```

EmailNotificacion.java

```

public class EmailNotificacion implements Notificacion {
    @Override
    public void enviar(String mensaje) {
        System.out.println("Enviando email: " + mensaje);
    }
}
  
```

```
}  
}
```

SmsNotificacion.java

```
public class SmsNotificacion implements Notificacion {  
    @Override  
    public void enviar(String mensaje) {  
        System.out.println("Enviando SMS: " + mensaje);  
    }  
}
```

WhatsappNotificacion.java

```
public class WhatsappNotificacion implements Notificacion {  
    @Override  
    public void enviar(String mensaje) {  
        System.out.println("Enviando WhatsApp: " + mensaje);  
    }  
}
```

TelegramNotificacion.java

```
public class TelegramNotificacion implements Notificacion {  
    @Override  
    public void enviar(String mensaje) {  
        System.out.println("Enviando Telegram: " + mensaje);  
    }  
}
```

PushNotificacion.java

```
public class PushNotificacion implements Notificacion {  
    @Override  
    public void enviar(String mensaje) {  
        System.out.println("Enviando Push: " + mensaje);  
    }  
}
```

Ejemplo de ejecucion:

```
public class Main {
    public static void main(String[] args) {
        System.out.println("--- Sistema Original ---");

        // Usando Email
        Notificacion email = new EmailNotificacion();
        GestorTurnos gestor1 = new GestorTurnos(email);
        gestor1.confirmarTurno("Su turno ha sido confirmado para
el lunes.");

        // Usando SMS
        Notificacion sms = new SmsNotificacion();
        GestorTurnos gestor2 = new GestorTurnos(sms);
        gestor2.confirmarTurno("Recordatorio de turno mañana.");

        System.out.println("\n--- Nuevos Canales (Extensibilidad)
---");

        // Usando Telegram
        Notificacion telegram = new TelegramNotificacion();
        GestorTurnos gestor3 = new GestorTurnos(telegram);
        gestor3.confirmarTurno("Turno agendado exitosamente.");

        // Usando Push
        Notificacion push = new PushNotificacion();
        GestorTurnos gestor4 = new GestorTurnos(push);
        gestor4.confirmarTurno("Tiene un nuevo mensaje del
doctor.");
    }
}
```